

IMPLEMENTATION & VERIFICATION GUIDE · V2

Meetsy v2

Reorganization, evidence-first review, learning-loop trust, KB legibility, and per-workspace tuning — the cockpit rebuild that finally makes the meeting-intelligence engine legible to IC engineers.

Full walkthrough of Phases 0 → 6, per-route UI verification journey, and success-criteria checklist · All six phases complete & verified

Generated 2026-07-19 · Branch `feat/meetsy-phase0` · Last commit `7884b32` · Phase 6 UX polish landed

Monorepo · Clicksy (root) + Meetsy (`apps/meetsy-api` , `apps/meetsy-web` , `packages/*`)

Contents

- 1 v2 in one page — the four problems it solves
 - 2 Audience decision + the six phases
 - 3 System architecture & the dependency graph
 - 4 Phase 0 — Foundations
 - 5 Phase 1 — IA + Home + History
 - 6 Phase 2 — Evidence-first review + push retry
 - 7 Phase 3 — Learning trust
 - 8 Phase 4 — `/kb` consolidation
 - 9 Phase 5 — `/tuning` per-workspace tunables
 - 10 Phase 6 — Cross-cutting UX polish
 - 11 Data model changes at a glance
 - 12 End-to-end sequence (upload → push)
 - 13 Queues, streams & data stores
 - 14 UI journey — step-by-step verification
 - 15 Success-criteria checklist
 - 16 Verification results & ops appendix
-

1 · v2 in one page — the four problems it solves

v1 (Phases 0–3 of the `2026-06-27` integration plan) shipped the **engine**: shared cookie-session auth, ClickUp write-back, RAG knowledge base, smart-assign + a learning loop. Every task on a run carried three predictions (sprint / duplicate / owner) and a nudge chip when the learning gate fired. But four things blocked trust and adoption — and v2 (the `2026-07-18` plan) is the cockpit rebuild that fixes them.

Problem 1 · The review page does everything

`apps/meetsy-web/app/runs/[runId]/*` was ~1200 lines rendering stepper + tasks + push editor + chat + learning in one scroll. No home, no history, no way back to a past run except URL memory.

v2 fix → Phase 1 introduces `/home`, `/meetings`, full-text search, tabbed `/runs/:id` (Overview / Push / Chat / Insights).

Problem 2 · Evidence is computed but not shown

`assignment.ranked[]`, `evidenceTaskIds` (5 precedent tasks per candidate), `FieldPrediction.candidates[]`, `kbContext.sourceId` — all in the payload, none rendered. Every task-id chip was dead text.

v2 fix → Phase 2 rewrites `signals.tsx` so evidence is expanded by default; every chip is a clickable button that opens a side sheet with the ClickUp task's title/status/owner/URL.

Problem 3 · The learning loop is invisible until it fires

The panel at `components.tsx:1021` filtered `gatePassed===true` and dropped the tail. Users couldn't see a pattern building up (2/3 corrections in). Metric labels used jargon ("*Raw model accuracy proxy*").

v2 fix → Phase 3 adds `/learning` (Active / Building up / Coverage), an SSE near-gate toast, `FIELDS = ["assignee", "sprint"]` expansion, plain-English metric labels.

Problem 4 · The KB is legible only during onboarding

`GET /workspaces/:id/kb/search` (hybrid RRF) and `/kb/documents/:docId` existed with no UI. Nothing to browse or inspect what the model knows about your workspace.

v2 fix → Phase 4 consolidates everything under `/kb` — Overview / Tasks / Documents / Search / Rebuild — plus a global `⌘K` palette that searches the KB from any route.

Plus a silent-loss bug (Phase 0 R1/R2): `AnalysisRun.result` 's five signal keys (`kbContext` , `fieldPredictions` , `duplicates` , `assignment` , `adjustments`) were stripped by `analysis.service.ts:227` on every feedback submit and every chat-added task — so a run's evidence disappeared the first time it was touched. v2 Phase 0 introduces `ReviewResultSchema` in `@ma/shared` + a `mergeSignals()` helper called before every persist. All five keys (plus the new `neighboursByTask` from Phase 2) now round-trip.

WHAT A V2 USER GETS, IN ONE SENTENCE

Land on `/home` after login → see recent runs and a personal "is-the-model-getting-better-at-predicting-me" digest → open any past run without URL memory → see evidence expanded on the task cards → click any chip to see the actual ClickUp task → watch the learning loop *build up* instead of just firing → tune per-workspace thresholds with preview replay → and drive it all keyboard-first, dark-mode-aware, on any screen size.

2 · Audience decision + the six phases

The single locked audience decision for v2, made 2026-07-18 with the product owner: **IC engineers checking their assignments**. Every downstream choice follows from it.

What "IC engineers as primary user" changed:

- Evidence panels default to **expanded**, not collapsed — an IC wants to see the reasoning without an extra click.
- Every provenance chip (`CU-01H...`) is **clickable** → **side sheet** with title/owner/status/link.
- Review page gets **keyboard traversal** (`j` / `k` between tasks, `Esc` closes the sheet).
- Push editor becomes a **secondary tab**, not the default view. Overview leads.
- Nudges are framed as **"here's what your team taught the model"**, not "click accept to be faster."

The six phases at a glance

#	Phase	Delivers	Backend footprint
0	Foundations	Signal-loss fix · <code>WorkspaceMLConfig</code> + <code>AnalysisRunSnapshot</code> migrations · task-lookup + runs-list endpoints · <code>ReviewResultSchema</code> in <code>@ma/shared</code> · <code>shadcn/ui</code> + <code>lucide</code> + <code>next-themes</code> + <code>sonner</code> .	~4 endpoints, 2 tables, 1 shared schema.
1	IA + Home + History	Left sidebar, <code>/home</code> , <code>/meetings</code> list with full-text search, <code>/runs/:id</code> re-scaffolded into tabs. Per-user learning digest.	<code>GET /workspaces/:id/learning/me</code> , <code>GET /workspaces/:id/runs/search?q</code> , <code>tsvector</code> column + GIN index.
2	Evidence-first review	Task cards with expanded evidence, every chip clickable → side sheet, raw top-5 kNN neighbours per task, push-retry queue + dead-letter.	Push-retry BullMQ queue + <code>PushDeadLetter</code> table + <code>POST /runs/:id/push/retry</code> .
3	Learning trust	<code>/learning</code> (Active / Building up / Coverage), gate constants in API, renamed jargon metrics, <code>FIELDS = ["assignee", "sprint"]</code> , near-gate SSE toast.	Pattern-history endpoint, Redis learning cache, learning SSE channel.
4	<code>/kb</code> consolidation	Single <code>/kb</code> route (Overview / Tasks / Documents / Search / Rebuild). Global <code>%K</code> palette. <code>/onboarding</code> retired.	<code>GET /workspaces/:id/kb/tasks?cursor&filter</code> .
5	<code>/tuning</code> (Owner)	Per-workspace tunables (dup bands, sim floor, gate values, model routing). Preview endpoint replays last-N runs against candidate config.	<code>GET PUT /workspaces/:id/ml-config</code> , <code>POST /workspaces/:id/ml-config/preview</code> .
6	Cross-cutting UX	Dark-mode palette sweep + toggle, keyboard shortcuts + landmarks, empty states + skeleton loaders, <code>prefers-reduced-motion</code> , mobile-safe review.	None — frontend-only.

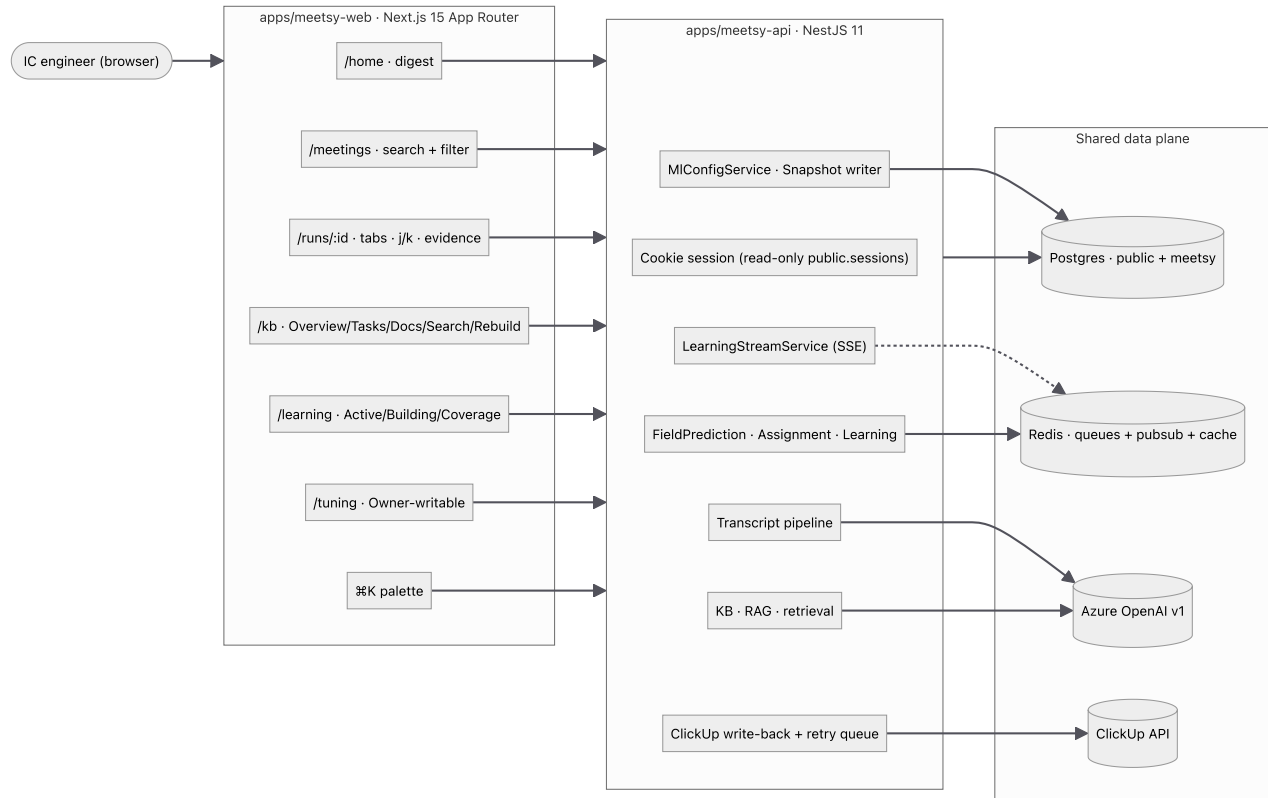
Delivery unit

Each phase was split into a small number of PRs (labelled A → DD). Every phase closed with a **single atomic commit** and a verify table in `docs/meetsy/BUILD-JOURNAL.md` (typecheck + lint

+ jest suite). Migrations were hand-authored and stay unapplied at commit time — the orchestrator applies via `prisma migrate deploy`.

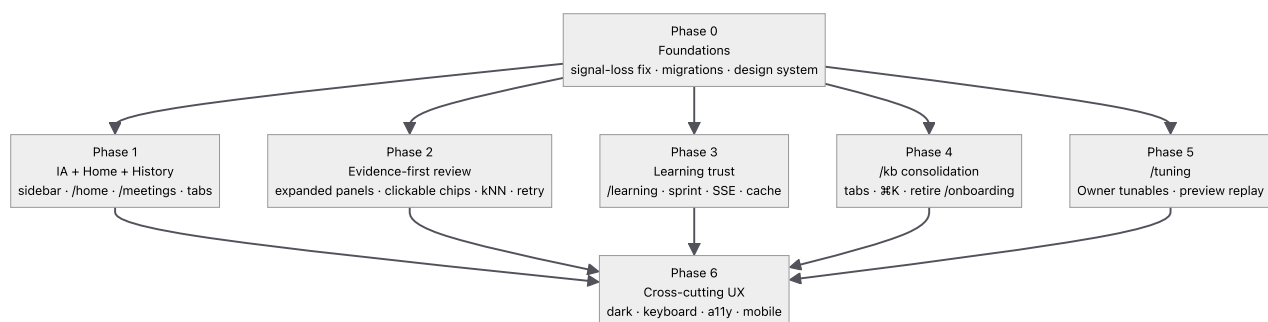
3 · System architecture & the dependency graph

Meetsy is a separately-deployable sibling to Clicksy. Shared: login/session, org/workspace model, one Postgres (Meetsy owns `meetsy` schema, reads Clicksy's `public` read-only), one ClickUp token, one Redis instance. Meetsy adds an Azure OpenAI dependency for the pipeline (chat + embeddings on the unified `niftyaibd-resource.services.ai.azure.com/openai/v1` endpoint as of 2026-06-29).



Phase dependency graph

Phase 0 unblocks everything. Phases 1/2/3 fan out from it. Phases 4 and 5 depend on Phase 0 alone (KB endpoints + `WorkspaceMlConfig` already exist). Phase 6 is a cross-cut polish that touches every route from Phases 1–5.



What DID and DIDN'T change under the hood

Changed (v2)

- Result-persist path rewired to preserve five signal keys + `neighboursByTask` .
- Two new meetsy tables (`WorkspaceMlConfig` , `AnalysisRunSnapshot`) + a dead-letter table + a tsvector column on `Meeting` .
- Web routing collapsed: `/` → `/home` , `/onboarding` → `/kb` , `/settings/kb` → `/kb?tab=rebuild` .
- Learning gate constants (`MIN_CORRECTIONS` , `MIN_AGREEMENT`) sourced from `WorkspaceMlConfig` ; duplicate bands too.
- SSE learning-stream via Redis pub/sub, mounted at the shell.

Deliberately did NOT change

- The engine itself. Same pipeline, same retrieval, same smart-assign math from v1 Phase 3.
- Clicksy's `src/` . All Clicksy-side edits (v1) were the two cookie-Domain lines.
- Session shape. Same `clickup_sync_sid` cookie, same CSRF double-submit.
- Model choice — **gpt-5.5** stays the pipeline main; overrides land in `WorkspaceMlConfig.models` but are stored, not yet consumed (deferred).

4 · Phase 0 — Foundations

Goal: unblock every subsequent phase in one PR triple. Shipped as PR-A (signal-loss fix), PR-B (Prisma models + endpoints), PR-C (design-system foundations).

PR-A · signal-loss fix

Root cause (grounded in v2 spec §1.1): `loadRunContext` at `analysis.service.ts:227` parsed `run.result` with strict `AnalysisResultSchema.parse()` , dropping the five signal keys. `assemble()` then rebuilt a strict `AnalysisResult` without them. Both feedback and chat writes persisted the stripped payload. As a knock-on, `push.service.ts:158` read `.fieldPredictions` off the raw JSON blob and returned `{}` — the `FieldOverride` logger silently died for those runs.

Fix:

- New `packages/shared/src/review-result.ts` — Zod source-of-truth for `KbContextHitSchema` , `TaskPredictionSchema` (with `FieldPredictionSchema` , `PriorCandidateSchema` , `DuePredictionSchema`), `DuplicateHitSchema` ,

`TaskAssignmentSchema` , `TaskAdjustmentsSchema` . Every signal key is **optional** so historical runs still parse.

- `ReviewResultSchema` re-exported and swapped into `RunResponseSchema.result` , `SubmitFeedbackResponseSchema.result` , `SendChatResponseSchema.result` .
- New `mergeSignals(base, source)` helper called from **both** write paths (`submitFeedback` line 291, `sendChat` line 366) — the merged `ReviewResult` , not the stripped `AnalysisResult` , is what gets persisted and returned.
- Web: local `ReviewResult` interface in `signals.tsx` (77 lines) deleted; now imports from `@ma/shared` .

PR-B • Prisma models + endpoints

WorkspaceMLConfig		
string	workspaceId	PK
string	orgId	
json	tunables	
json	models	
string	updatedBy	
datetime	updatedAt	
datetime	createdAt	

AnalysisRun		
string	id	PK
string	workspaceId	
string	status	
json	result	
datetime	createdAt	

1:0..1 cascade

AnalysisRunSnapshot		
string	runId	PK
string	workspaceId	
json	tunables	
json	models	
datetime	createdAt	

- **New model** `WorkspaceMLConfig` — `workspaceId` PK (one row per workspace), `tunables` JSONB, `models` JSONB, `updatedBy?` , index on `orgId` . Soft ref to `public.workspaces` .
- **New model** `AnalysisRunSnapshot` — 1:1 with `AnalysisRun` via cascade FK, frozen `tunables` + `models` . Append-only. Enables reproducible `/tuning` preview in Phase 5.

- **New index** on `AnalysisRun(workspaceId, createdAt)` — powers the newest-first paginated runs-list.
- **New GET `/workspaces/:id/clickup/tasks/:taskId`** — resolves ClickUp task ids surfaced in duplicate chips + `evidenceTaskIds` to `{title, status, assigneeName, url}`. Soft-scopes by workspace; returns `null` on soft-deleted tasks so the chip degrades to "unavailable."
- **New GET `/workspaces/:id/runs?limit&offset&status`** — paginated list with a derived `pushStatus` : `not_configured` / `not_pushed` / `partial` / `pushed` . Batches one `$transaction([findMany, count])` plus a page-wide `TaskPush.findMany` .

PR-C · design-system foundations

Adds shadcn/ui + Radix primitives + lucide-react + next-themes + sonner to `apps/meetsy-web` . Sixteen primitives installed (button/card/dialog/sheet/tabs/dropdown-menu/command/tooltip/skeleton/sonner/input/select/label/checkbox/radio-group/separator). Existing `app/ui.tsx` becomes a one-line shim that re-exports from `app/ui-legacy.tsx` so no caller migration is forced — later phases opt in.

Tailwind v4 wiring (critical): `globals.css` uses `@custom-variant dark (&:where(.dark, .dark *));` (v4's way to enable class-based dark mode), `@import "tw-animate-css";` (Radix uses `animate-in/out` classes), and the shadcn oklch token palette in `:root + .dark` with an `@theme inline` block mapping vars to Tailwind utility names.

PHASE 0 VERIFY (ALL GREEN)

Target	Result
<code>@ma/shared build</code>	PASS
meetsy-api typecheck	PASS
meetsy-web typecheck + lint	PASS (0/0)
meetsy-api jest	41 suites / 235 tests
Clicksy backend tests	108 suites / 824 tests (no regression)

5 • Phase 1 — IA + Home + History

Goal: collapse the single-route review page into a sidebar + home + history layout. Shipped as PR-D (sidebar + `/home` + `/meetings`), PR-E (full-text search backend + UI), PR-F (per-user learning digest), PR-G (`/runs/:id` tabs).



PR-D • sidebar shell + `/home` + `/meetings`

- **New** `components/nav/sidebar.tsx` — a persistent left rail on `md+`; smaller screens collapse into an off-canvas `Sheet` triggered by a hamburger. Active-route highlight matches the current pathname's PREFIX (`/settings/kb/documents` still highlights KB).
- `/` → **redirect to** `/home`. The old upload form moved to `/new/page.tsx` via `git mv` (history intact).
- `/home` — two-column layout: recent-runs list (limit 5) on the left, learning-digest slot on the right. Shared `RunRow` component.
- `/meetings` — paginated full history with filter chips (All / Completed / Running / Failed), `?q=&page=&status=` URL surface, wrapped in `<Suspense>` per Next 15's `useSearchParams` rule.

PR-E • runs full-text search

- **Hand-authored migration** adds a generated `tsv tsvector` column to `meetsy."Meeting"` (title weighted A, transcript weighted C) + a GIN index. Postgres 12+ generated column recomputes on every INSERT/UPDATE — no trigger, no backfill.
- `AnalysisService.searchRuns` uses `$queryRaw` for the WHERE + rank + count (order `ts_rank_cd DESC, createdAt DESC`), reuses the same push-status derivation as `listRuns`. Returns the exact `RunListView` shape.
- `GET /workspaces/:id/runs/search?q=&limit=&offset=&status=` — empty/whitespace `q` → 400; `q > 200` chars → 400. Declared BEFORE `GET /runs/:id` so the route isn't shadowed as a run id.
- `/meetings` **search UX**: 300ms-debounced input; commit on debounce via `router.replace` so filter churn doesn't fill history.

PR-F • per-user learning digest

- `LearningService.meSummary(workspaceId, userId)` joins `FieldOverride` ↔ `TaskPush` on `(runId, meetsyTaskId)` and filters by `TaskPush.pushedBy = userId`. Buckets the last 6 ISO weeks (Monday UTC), zero-padded so a sparkline has a fixed x-axis.

- `GET /workspaces/:id/learning/me` — any authenticated user; the `me` in the path is a literal, the `userId` comes from the session principal.
- `components/learning/digest-card.tsx` — one card, three metric rows (Accuracy, Corrections, Nudge acceptance), each with a hand-drawn SVG `Sparkline` (no `chart-lib` dep). Empty state ("As you review runs...") when `totalOverrides = 0`.

PR-G • `/runs/:id` tabbed scaffold

- `ResultsSection` / `PushSection` / `ChatPanel` split into **Overview** / **Push** / **Chat** tabs plus a placeholder **Insights**.
- `forceMount` on every `TabsContent` with `data-[state=inactive]:hidden` — Radix would otherwise unmount inactive panels, restarting `ChatPanel` state + `PushSection` fetches.
- **Hash sync:** initial tab reads from `#hash` (deep-linkable — `/runs/abc#push` opens on Push). Tab clicks use `history.replaceState`.

PHASE 1 VERIFY

Target	Result
meetsy-api jest	43 suites / 246 tests (+11)
meetsy-web typecheck + lint	PASS (0/0)
Clicksy tests	110 suites / 835 tests (unchanged)

6 • Phase 2 — Evidence-first review + push retry

Goal: make the review page the answer to "why did the pipeline think that?" — every signal the API computes is rendered inline, expanded by default, with drill-down. Shipped as PR-H (kNN neighbours), PR-I (push retry queue + dead-letter), PR-J (evidence panels), PR-K (chip → side-sheet + retry-failed button).

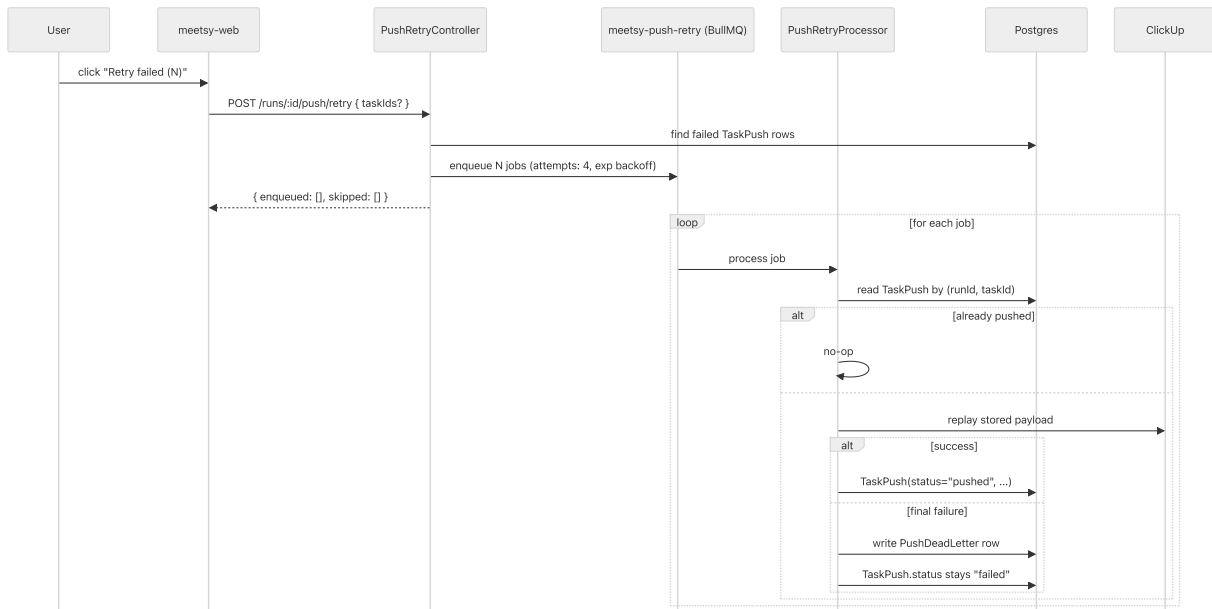
PR-H • top-5 kNN neighbours attached to `run.result`

The kNN neighbours per task were already computed by `FieldPredictionService` and threaded through the processor for owner ranking — then dropped on the floor at `persist`. PR-H adds `neighboursByTask` so the review UI can show "why did the pipeline think of this task as similar?" without re-embedding.

- **Shared schema:** `NeighbourHitSchema` in `@ma/shared` — mirrors the runtime `Neighbour` interface with `createdDate` / `closedDate` as `z.string().datetime().nullable()`.
- **Persist:** the `result: {...}` write at `analysis.processor.ts:243` gains `neighboursByTask: sliceNeighbours(taskAnalysis.neighboursByTask, 5)`.

- **mergeSignals():** extended to preserve `neighboursByTask` — same failure mode Phase 0 R2 fixed for the other signal keys.
- **Bounds:** top-5 × ~20 tasks × ~100 bytes ≈ 10 KB/run — trivial vs. transcript-sized fields already on `run.result`.

PR-I · push retry queue + dead-letter



- **Hand-authored migration** adds `meetsy."PushDeadLetter"` table with `{id, runId, meetsyTaskId, workspaceId, jobId, payload, errorMessage, errorStack, attemptsMade, failedAt, retriedAt, resolvedAt, resolvedBy}` + 2 indexes.
- **Queue name** `meetsy-push-retry`. Job id `${runId}:${meetsyTaskId}:${nonce}` (nonce is `Date.now().toString(36)` — per-enqueue so a retry-after-retry actually runs; the DB row is the idempotency key). Options: `attempts: 4, backoff: { type: "exponential", delay: 2000 }`.
- **Retry endpoint** `POST /runs/:id/push/retry` — body `{ taskIds?: string[] }`. Response `{ enqueued: [], skipped: [{ meetsyTaskId, reason }] }`. Reasons: `not_found, not_failed:<status>, enqueue_failed:<msg>`.
- **Dead-letter admin** `PushDeadLetterController` at `/workspaces/:id/push/dead-letter` — Owner/Admin only; GET lists unresolved by default, POST `/:deadLetterId/resolve` hand-marks with `resolvedBy = user.userId`.

PR-J · evidence panels expanded by default

`signals.tsx` rewritten from the ground up. The old file rendered four shallow rows (duplicates → 3 pred chips → optional nudge → owner name-only) with `assignment.ranked[]`, `FieldPrediction.candidates[]`, and `evidenceTaskIds` never rendered.

Sections rendered per task card (each short-circuits when its signal is absent):

1. **Duplicates** — every `d.taskId` renders as a `TaskChip` (red/amber by band).
2. **Suggested fields** — Sprint / Due / Estimate chips PLUS a sub-strip of sim-weighted candidates (from A · 42% · B · 25% ...), picked value styled distinct. A `clamp` badge appears when `isModal === false` (LLM overrode the modal top).
3. **Owner** — top-3 ranked candidates with score bars (`ownershipScore`), closed/open/tracked-hours line, plus every `evidenceTaskIds` (up to 5) as clickable `TaskChip` s. "Show N more" expands to the full ranking; recommended candidate highlighted green.
4. **Learning nudge** — `assignee` AND `sprint` (Phase 3 fills the sprint side of `TaskAdjustments`).
5. **Similar (neighboursByTask)** — top-3 neighbours as `TaskChip` s with the cosine %.

PR-K · clickable chips → side sheet + retry-failed button

- **TaskSheetProvider** — a single side-sheet slot per view; any chip in the subtree calls `openTaskSheet(taskId)` . Scoped to `runs/[runId]/page.tsx` in Phase 2; hoisted to `AppShell` in Phase 4 so the KB tab can reuse it.
- **TaskChip** — a real `<button>` , 6 tone variants, focus ring for keyboard traversal. Falls back to a no-op when no provider is mounted.
- **TaskDetailSheet** — right-side `Sheet` , re-fetches on each open via `api.getClickupTask(workspaceId, taskId)` . Three legit states: loading, loaded (title/status/assignee/updatedAt/"Open in ClickUp ↗"), null (task predates KB — legitimate; message reassures the chip is still trustworthy).
- **Retry-failed button** in `PushSection` : `variant="secondary"` "Retry failed (N)" that fires `api.retryFailedPushes(runId)` and reloads after 3s. Only rendered when `failedCount > 0` .

PHASE 2 VERIFY

Target	Result
meetsy-api jest	46 suites / 262 tests (+16)
meetsy-web typecheck + lint	PASS (0/0)
New migration	<code>20260718200000_meetsy_v2_phase2_push_dead_letter</code> · UNAPPLIED at commit

7 · Phase 3 — Learning trust

Goal: make the loop legible and trustable. Before Phase 3 the loop's state was hidden in a single settings panel; users saw a nudge or didn't, with no signal that they were "one correction away" from teaching the loop. Shipped as PR-L (`FIELDS` expansion), PR-M (Redis cache + gate + history), PR-N (near-gate SSE), PR-O (`/learning` page).

PR-L · `FIELDS = ["assignee", "sprint"]`

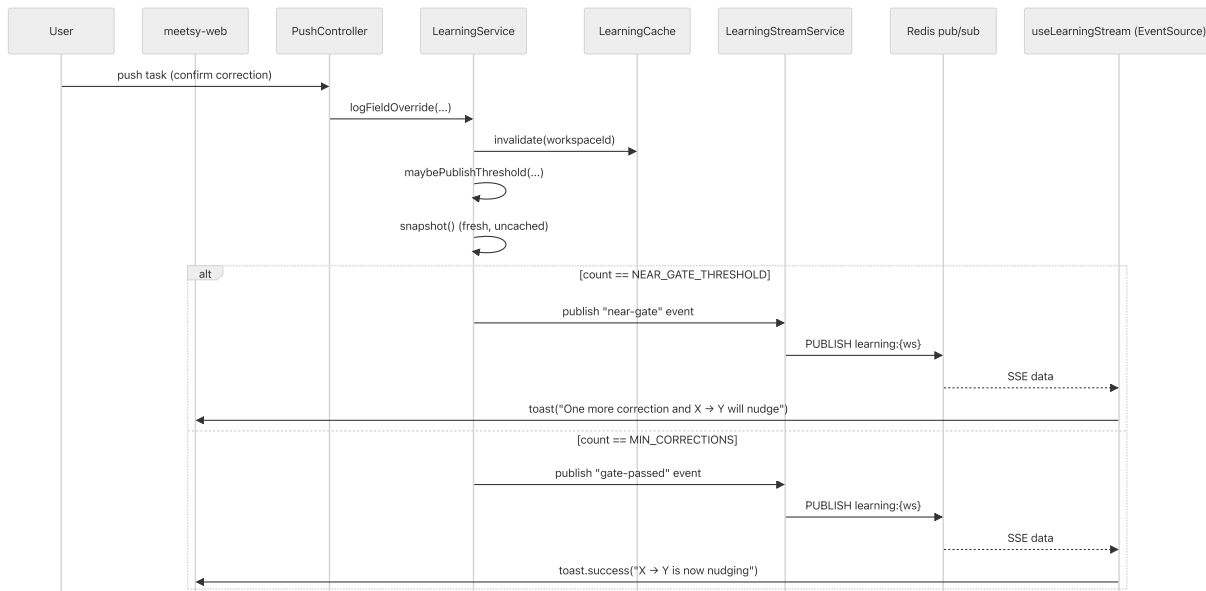
What "expand the learning loop" means concretely: before v2, `LearningService` was hardcoded to one field. Snapshot type was `{assignee: FieldAggregate}`, `TaskAdjustments` was `{assignee?: ...}`, push only branched on the assignee prediction. Phase 3 widens to a second learnable field (`sprint`) with the discipline that adding a THIRD field in a future phase is a five-line change.

- `CorrectionStat` gains `field: string` (which learnable field) and `key: string` (URL-safe base64url of `field|predicted|confirmed`, used as the path segment on `/learning/patterns/:key/history`).
- **Sprint resolution asymmetry** (documented): assignee resolves `clickupUserId → name`, sprint resolves via `WorkspacePushConfig.sprintLists[]` (`listId → name`). Both happen at aggregate time.
- Existing spec suite ported — every `aggregateField(...)` call now passes `"assignee"` first arg. New `learning.service.sprint.spec.ts` proves the sprint path end-to-end.

PR-M · Redis snapshot cache + gate endpoint + pattern history

- `LearningCacheService` — Redis KV cache for `LearningService.snapshot()`. Key `meetsy:learning:snapshot:v1:{workspaceId}`, SETEX 3600. Hard-invalidated on `logFieldOverride`. The `v1` guards against a future snapshot-shape change reading a stale value. Read-through-cached: hit returns stored value; miss falls through to DB scan then writes back.
- `GET /workspaces/:id/learning/gate` — returns `{minCorrections, minAgreement, nearGateThreshold, fields}`. Workspace-independent today; Phase 5 makes it per-workspace by reading from `WorkspaceMlConfig` (shape stays stable).
- `GET /workspaces/:id/learning/patterns/:key/history` — decodes the base64url key (400 malformed / unknown field), consults the resolved snapshot for stats (404 if the pattern isn't in the workspace), then scans up to 500 newest `FieldOverride` rows and filters to those matching (`predicted`, `confirmed`). `nudgeShown` per entry so the UI can badge nudge-influenced rows.

PR-N · near-gate SSE toast



- **LearningStreamService** — Redis pub/sub for workspace-scoped `near-gate` | `gate-passed` events. Mirrors `kbChannel` : dedicated publisher connection, per-subscription subscriber connection, teardown on client disconnect.
- **classifyThreshold(count, minCorrections)** — pure decision, `NEAR_GATE_THRESHOLD` → "near-gate", `MIN_CORRECTIONS` → "gate-passed", else `null`.
- **Skips (belt-and-braces):** agreement rows (`predicted === confirmed`), abstain/unresolved (no value on one side), and nudge-influenced writes (they don't count toward the organic aggregate per `learning-aggregate.ts:73`).
- **GET /workspaces/:id/learning/stream** (`@Sse`) — Observable returned SYNCHRONOUSLY with async workspace resolution inside; Nest's SSE handler subscribes to the return value and does not unwrap a Promise.
- **Web wire-up:** `lib/useLearningStream.ts` mounted inside `SignedInShell` so toasts fire workspace-wide.

PR-O · `/learning` workspace page

- **Active** — patterns the loop currently gates (`gatePassed === true`). Per row: `predicted → confirmed`, correction count, consistency %.
- **Building up** — near-gate patterns (`!gatePassed && count >= 1`). Per row: progress bar `count / minCorrections`, N of 3 label. Clicking opens the `PatternHistorySheet`.
- **Coverage** — per-field: predictions seen, "Predictions you changed" (formerly `rawOverrideRate`), "Suggestions shown" (`nudgeSample`), "Suggestions accepted", and — when > 0 — Unresolved (amber, so a resolution bug looks like a resolution bug, not "sparse data").
- **Metric renaming happens ONLY at this UI layer.** The API still returns `rawOverrideRate` / `nudgeAcceptanceRate`; the existing `LearningPanel` in the review page keeps working.

PHASE 3 VERIFY

Target	Result
meetsy-api jest	53 suites / 293 tests (+31)
meetsy-web typecheck + lint	PASS (0/0)
Migrations	None new — loop grew, Postgres shape didn't

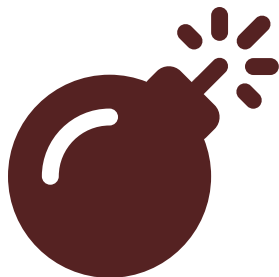
8 · Phase 4 — `/kb` consolidation

Goal: replace the seven-step `/onboarding` wizard and the deeply-buried `/settings/kb` with a single canonical `/kb` route, plus a global `%K` palette so KB search reaches the user from anywhere. Shipped as PR-P (`GET /kb/tasks`), PR-Q (`/kb` shell + 5 tabs), PR-R (retire `/onboarding`), PR-S (`%K` palette).

PR-P · `GET /workspaces/:id/kb/tasks`

- **KbTasksService** joins `KbChunk` (meetsy) with `public.clickup_tasks` and returns the distinct set of embedded ClickUp tasks, keyset-paged on (`updated_date DESC NULLS LAST, task_id DESC`).
- `?filter=<q>` narrows via a case-insensitive `ILIKE` against `task_name` , `client` , and `assignees_names` .
- **Cursor** is `base64url(JSON.stringify({u: iso|null, t: taskId}))` — malformed → 400.
- **Runs via** `prisma.$queryRaw` — Prisma's `groupBy` doesn't compose keyset paging + a distinct chunk count in one round-trip; all params bound through the tagged template.

PR-Q · `/kb` shell + Overview / Tasks / Documents / Search / Rebuild



Syntax error in text mermaid version 10.9.6

- Client route; shadcn `<Tabs>` keyed off `?tab=<name>` . Owner/Admin sees Rebuild; Members see the other four.
- **Status card** + idle banner render ABOVE the tabs so a not-ready KB is visible without a redirect (see PR-R).
- **Search tab:** plain `Input` with 300ms debounce → `api.kbSearch(ws, q, 20)` . Hits render as clickable cards; clicking opens the shared `TaskDetailSheet` . `?q=` echoed back into the

URL so browser back is honest.

PR-R · retire `/onboarding`

- `KbGate` deleted (v1's redirect-to- `/onboarding`). `SignedInShell` renders children directly.
- `app/onboarding/page.tsx` deleted; `app/onboarding/steps.tsx` → `app/kb/steps.tsx` via `git mv` .
- `app/settings/kb/page.tsx` → server-side `redirect("/kb?tab=rebuild")` . External bookmarks resolve.
- **Idle-state UX:** when `kbStatus.status !== "ready"` , the page renders a banner above the tabs. Owner/Admin can flip it into a `KbBuildPanel` ; Members see a read-only "ask an Owner/Admin" note.

PR-S · global `%K` command palette

- **Toggled by** `%K` or `Ctrl+K` via a `window` keydown listener.
- **Two groups:** **Go to** (Home, New meeting, Meetings, Learning, Knowledge base; Owner/Admin also sees Push settings) and **Search knowledge base** — 250ms-debounced live search against `/kb/search` (`k=8`).
- **Selecting a hit** navigates to `/kb?tab=search&q=<query>` (the palette closes; the search tab renders the full hit list in context).
- **Mounted in** `AppShell.tsx` next to the `<Toaster>` — inside `SignedInShell` so `useWorkspace` is safe, outside `<main>` so it overlays the whole app.

PHASE 4 VERIFY

Target	Result
meetsy-api jest	54 suites / 300 tests (+7)
meetsy-web typecheck + lint	PASS (0/0)

9 · Phase 5 — `/tuning` per-workspace tunables

Goal: close the gap Phase 0 opened. Phase 0 landed the `WorkspaceMlConfig` + `AnalysisRunSnapshot` tables and a `MlConfigService.forWorkspace(...)` accessor, but nothing in the runtime actually READ those tunables — the analysis pipeline still used compile-time constants. Phase 5 wires the two tunable groups whose consumers are already isolated behind pure functions (**dup bands + learning gate**) and adds an Owner-visible `/tuning` page with a preview replay endpoint.

Honest scope note. The remaining tunables (`simFloor` / `minQualifying` / `closedWeight` / `rrfK` / `novelMaxSimCutoff` / `linkMinSim` / `embedBatch` / `model-routing`) are STORED in `WorkspaceMlConfig.tunables` but not yet consumed by runtime code — each requires a small refactor of its caller. The UI flags them with a "Not applied yet" chip so an Owner isn't misled. This is Phase 5's design decision, not a bug.

PR-T · `MlConfigService.upsert` + `GET | PUT /workspaces/:id/ml-config`

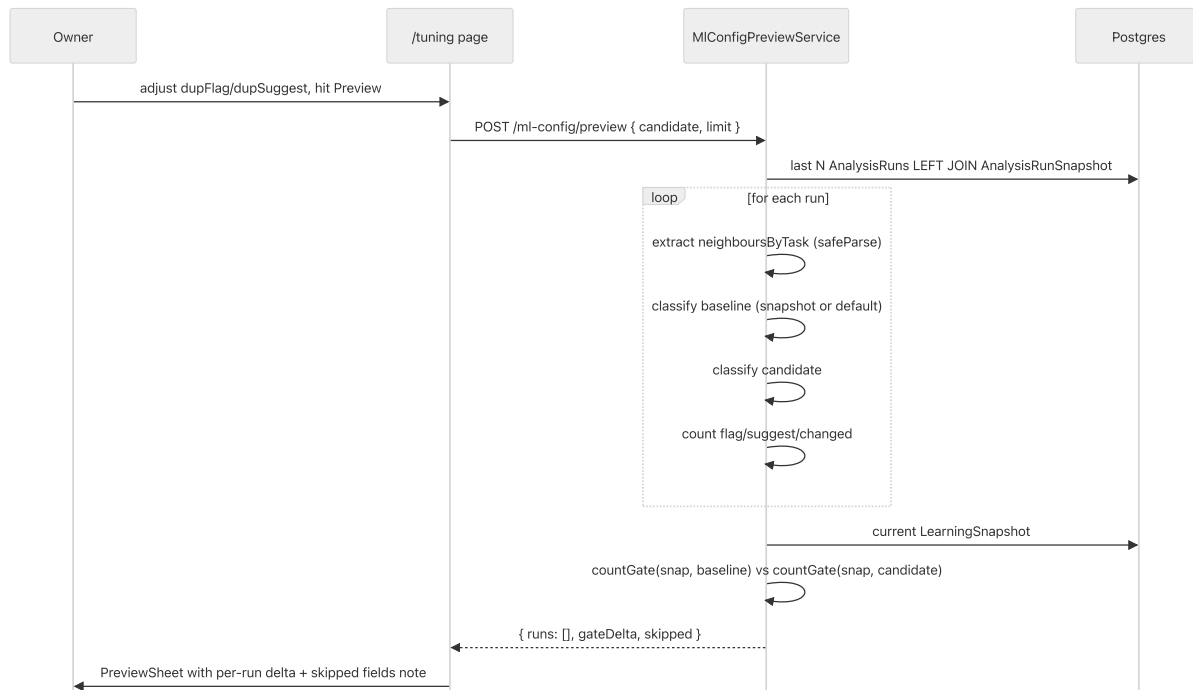
- `MlConfigService.viewForWorkspace(workspaceId)` returns `{tunables, models, updatedBy, updatedAt, isDefault}` — `isDefault=true` when the row is absent so the UI can show a "workspace hasn't customised this" chip.
- `MlConfigService.upsert(workspaceId, orgId, updatedBy, payload)` defensively re-parses through `RunSnapshotPayloadSchema` (belt-and-braces after the controller's `ZodValidationPipe`), does a single `prisma.workspaceMlConfig.upsert({...})`, then calls `this.cache.invalidate(workspaceId)` so the learning-snapshot cache doesn't return a stale value keyed off the pre-edit gate constants.
- **TuningController** : `GET` any authed user, `PUT` Owner-only, `POST /preview` Owner-only.

PR-U · runtime consumption of dup bands + learning gate

- `classifyDuplicates(neighbours, bands?)` — extended signature; `bands` defaults to compile-time constants so callers that don't pass one keep working.
- `FieldPredictionService.analyze(..., tunables?)` — optional so unit tests can keep calling the 3-arg form. When passed, plumbs `{dupFlag, dupSuggest}` to `classifyDuplicates`.
- **AnalysisProcessor** loads `const mlSnapshot = await this.mlConfig.forWorkspace(workspaceId)` up front, passes `mlSnapshot.tunables` to `fieldPrediction.analyze`, and REUSES the same snapshot for the `AnalysisRunSnapshot` write — one less query per run.
- `aggregateField(field, records, gate?)` — `gatePassed` derives from `gate.*`, so a workspace with `minCorrections=5` genuinely needs 5.

- `LearningService.gate(workspaceId)` is now async and reads from `mlConfig.forWorkspace(...).nearGateThreshold = max(minCorrections - 1, 0)` so a workspace tuned to `minCorrections=1` doesn't have a "near-gate" state.

PR-V • `POST /workspaces/:id/ml-config/preview`



- **Synchronous replay** — the spec's `meetsy-ml-preview` BullMQ queue is deferred; preview compute is cheap JSON math (a few kB of neighbour data per run × 10 runs).
- **Legacy runs** without `neighboursByTask` return `duplicates: null` so the UI can visibly say "this run pre-dates neighbour storage".
- **Gate delta is workspace-wide** — one `countGate(snap, baseline)` + one `countGate(snap, candidate)` against the current snapshot.
- `skippedFields()` returns the fixed list of non-replayable tunables + reasons for the UI's "these fields aren't in preview" note.
- **Limit** clamps to `[1, 20]` (default 10). Owner-only.

PR-W • `/tuning` web page

- Every `WorkspaceTunables` field renders as `<Input type="number">` bound to metadata in `tunable-meta.ts` (min/max/step from the Zod schema).
- Fields where `consumed=false` show a "Not applied yet" chip.
- Model routing renders as a read-only table (routing changes are a Phase 6+ refactor).
- **Preview** button opens a `PreviewSheet` with per-run duplicate deltas + workspace-wide gate delta + skipped-fields note.
- **Save** fires a `sonner` toast on success/failure.
- **Sidebar entry** — `Sliders` icon, `ownerAdminOnly: true`. Members can still navigate directly (read-only view); hiding the sidebar entry keeps their surface uncluttered.

PHASE 5 VERIFY

Target	Result
meetsy-api jest	56 suites / 313 tests (+13)
meetsy-web typecheck + lint	PASS (0/0)

10 · Phase 6 — Cross-cutting UX polish

Goal: a trust pass, not a feature phase. Phase 0 wired `shadcn/ui` + `next-themes` + `.dark` class variant, but no route was actually audited — 364 `zinc-*` / `slate-*` / `bg-white` usages across ~30 files meant `system=dark` rendered white cards on a dark background. Phase 6 closes that plus keyboard traversal, skeleton loaders, landmarks, focus rings, and `prefers-reduced-motion`. Backend footprint: **none**.

PR-AA · dark-mode palette sweep + theme toggle

- `components/theme-toggle.tsx` — `shadcn DropdownMenu` with Light / Dark / System items (Sun / Moon / Laptop icons). Trigger icon animates rotate/scale between Sun and Moon under the `.dark` class variant.
- **Sidebar** palette swept to semantic tokens (`bg-background` , `border-border` , `text-foreground` , `text-muted-foreground` , `bg-primary` , `bg-accent`). Active nav pill uses `bg-primary text-primary-foreground` . `aria-current="page"` on active link.
- `app/ui-legacy.tsx` primitives migrated to semantic tokens. Priority hues (red/orange/sky) kept but got dark-mode variants — legible in dark mode without changing meaning.
- **Bulk sweep across 22 application files:** `bg-white` → `bg-card` ; `text-zinc-{900,800,700}` → `text-foreground` ; `text-zinc-{600,500}` → `text-muted-foreground` ; `border-zinc-200` → `border-border` ; `bg-zinc-{100,200}` → `bg-muted` ; `hover:bg-zinc-100` → `hover:bg-accent` ; ...

PR-BB · keyboard shortcuts + landmarks + focus rings

- **Skip-to-content link** — first focusable element in `AppShell` : `sr-only focus:not-sr-only focus:fixed focus:top-2 focus:left-2 focus:bg-primary focus:text-primary-foreground focus:z-50` .
- `<main id="main-content" role="main" tabIndex={-1}>` — skip link + screen-reader landmark both target the same node. Redundant `role="main"` is belt-and-suspenders (some AT don't map the tag alone reliably).
- `useReviewKeys()` attaches a window-level `keydown` listener: `j` / `↓` focuses next `[data-task-anchor]` ; `k` / `↑` focuses prev; wraps at both ends; guards against firing while typing in `<input>` / `<textarea>` / `[contenteditable]` ; ignores keys with `Ctrl` / `Meta` / `Alt`

modifiers so `Ctrl+J` still works. Smooth-scrolls into view with

```
scrollIntoView({block:"nearest"}) .
```

- **TaskCard** wrapped in `<div data-task-anchor={task.id} tabIndex={-1} className="scroll-mt-24 rounded-xl focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2">` .
- **Focus-ring pass** — sidebar links + brand logo got `focus-visible:` rings. shadcn primitives already ship with rings via base recipes.

PR-CC · skeleton loaders + empty states

- `components/ui/empty-state.tsx` — dashed-border card, icon in a muted `rounded-full bg-muted` circle, title, optional description + single action. Semantic tokens throughout.
- `/home` : three `Skeleton` rows matching `RunRow` height. Hand-rolled "No runs yet" replaced with `EmptyState` pointing at `/new` .
- `/meetings` : five-row `Skeleton` stack. Two empty-state branches (search-no-results and filter-no-results) consolidated into one `EmptyState` that adapts icon (`Search` vs `ListChecks`) and action (Clear search vs Analyze a meeting).
- `/kb tasks tab`: five `Skeleton h-14` rows; `EmptyState` with `ListChecks` icon.
- **Button-embedded spinners kept** — `<Spinner>` inside `<Button>` is an affordance, not a layout loader.

PR-DD · reduced-motion + mobile-safe review

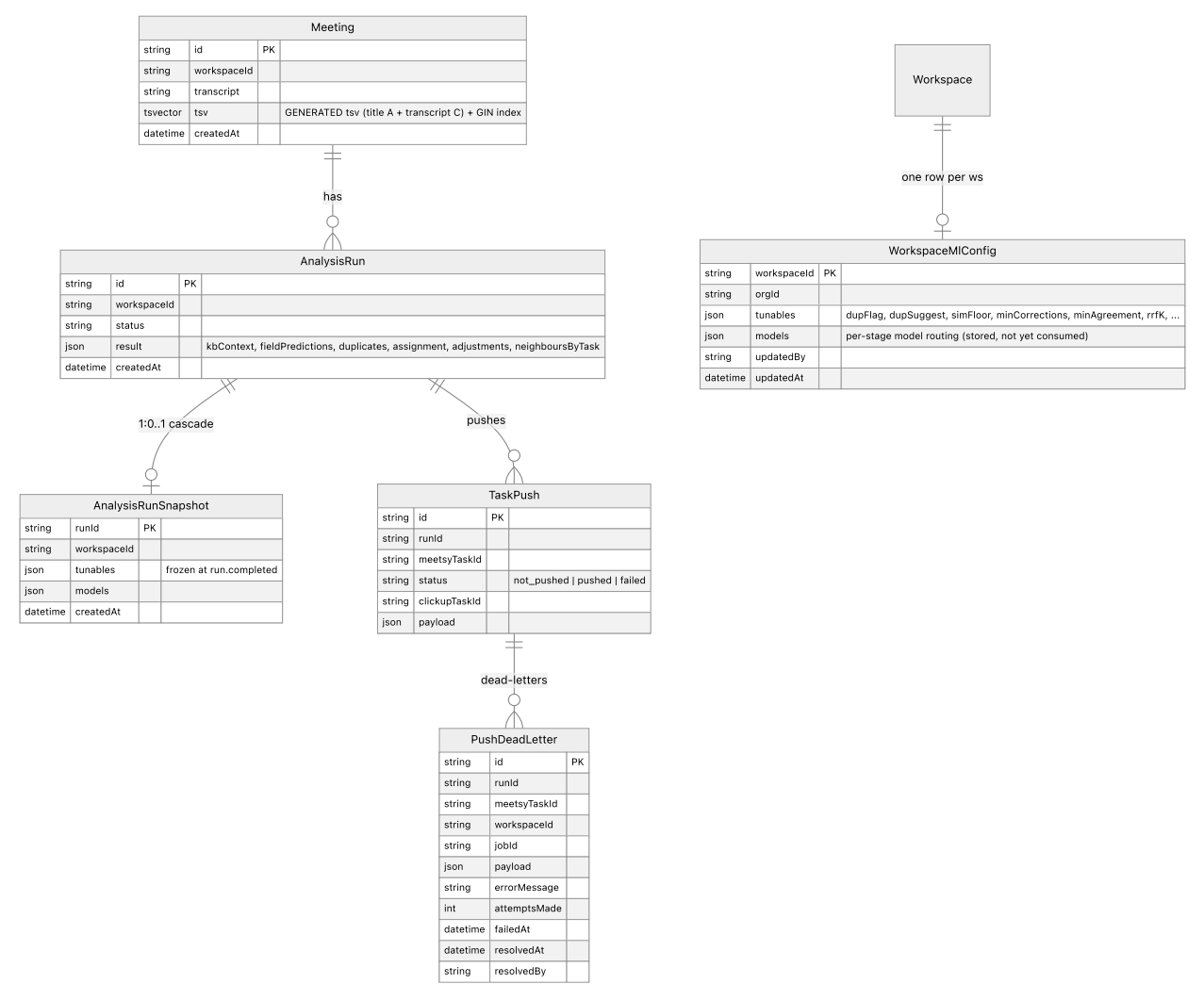
- `@media (prefers-reduced-motion: reduce)` block at `globals.css:105` : `animation-duration: 0ms !important; animation-iteration-count: 1 !important; transition-duration: 0ms !important; scroll-behavior: auto !important; on *, *::before, *::after` . Neutralizes shadcn slide/fade, sonner toast slide, sheet slides, Tailwind `animate-*` in one shot. `!important` because Tailwind utilities + Radix data-state animations have specificity we can't beat via cascade.
- **Mobile:** `PersonSection` 's task indent went from `pl-9` to `pl-4 md:pl-9` . On phones tasks stack flush-left instead of eating half the horizontal budget under the person's avatar column.
- **TaskDetailSheet** kept `side="right"` — shadcn's `SheetContent` already uses `w-full max-w-md` which collapses to full-width on phones (< 448px). Bottom-drawer flip was reconsidered and skipped (avoids a Radix Sheet unmount during resize).
- **aria-live** — `ErrorBanner` carries `role="alert"` ; Sonner's toast root declares `role="status"` .

PHASE 6 VERIFY

Target	Result
meetsy-web typecheck + lint	PASS (0/0)
meetsy-api jest (drift check)	56 suites / 313 tests (unchanged)

11 • Data model changes at a glance

v2 adds two tables + a dead-letter table + a tsvector column. Every migration is hand-authored, meetsy-schema only, and applied via `prisma migrate deploy` at deploy time.



Migration inventory

Timestamp	Phase	What it adds
20260718...	0	<code>WorkspaceMlConfig</code> + <code>AnalysisRunSnapshot</code> + <code>AnalysisRun</code> (<code>workspaceId</code> , <code>createdAt</code>) index.
20260718150000	1	<code>Meeting.tsv tsvector</code> generated column + GIN index. Postgres 12+ generated column recomputes on every INSERT/UPDATE — no trigger, no backfill.
20260718200000	2	<code>PushDeadLetter</code> table + 2 indexes.

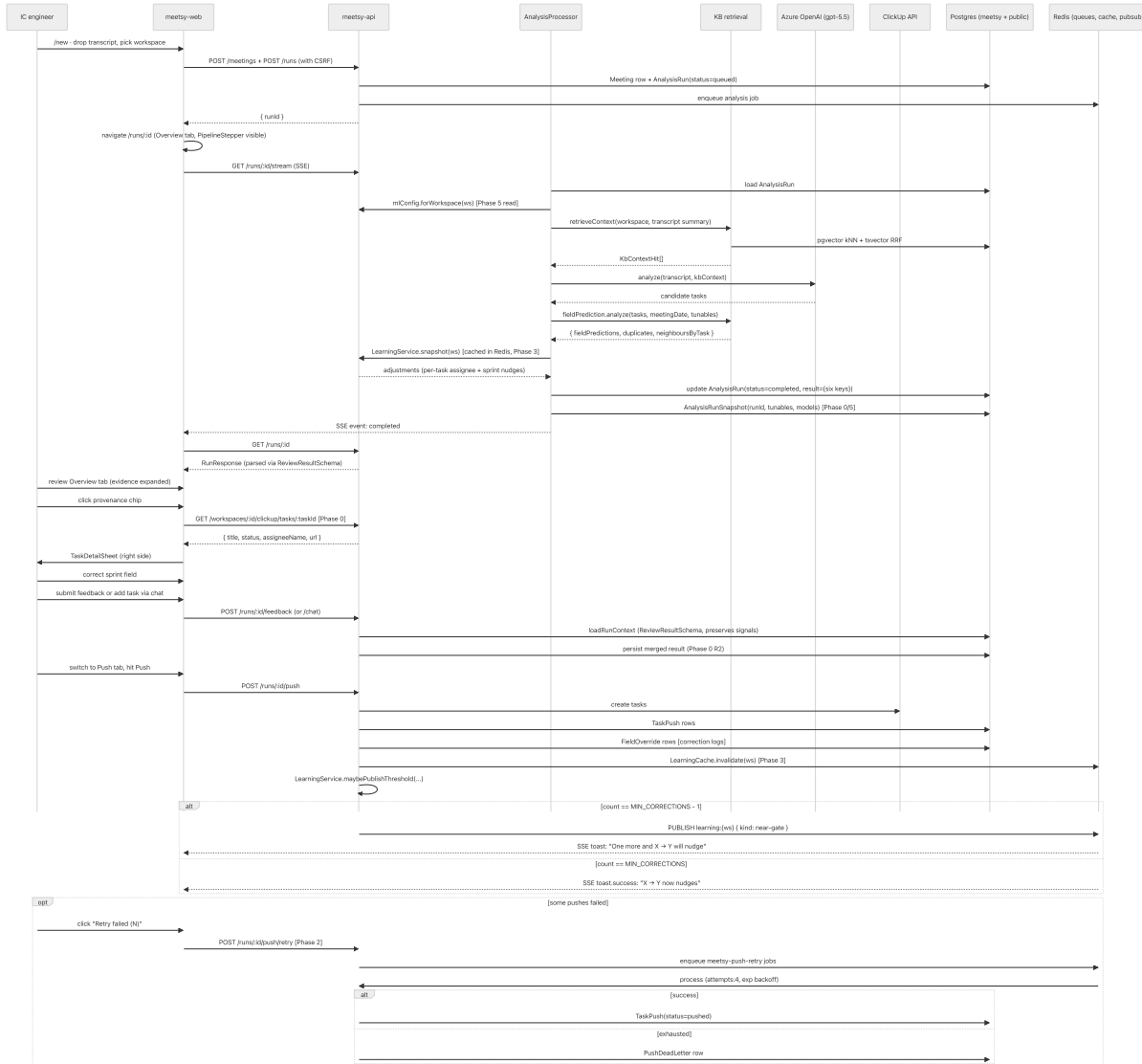
Signal keys on `AnalysisRun.result`

- `kbContext` — the ranked KB hits used at prompt time (workspace-level grounding).
- `fieldPredictions` — per-task `{sprint, due, estimate}` : modal value + sim-weighted candidates + priors.
- `duplicates` — per-task list of `{taskId, sim, band}` at the flag/suggest thresholds.
- `assignment` — per-task `{recommended, ranked[], reasoning}` with per-candidate `{ownershipScore, evidenceTaskIds}`.
- `adjustments` — per-task `{assignee?, sprint?}` — the "learning nudge" pulled from `LearningService.snapshot()`.
- `neighboursByTask` — **new in Phase 2** — top-5 kNN neighbours per task from `FieldPredictionService`. Cosine-descending, ~10 KB/run.

Round-trip invariant. All six signal keys survive `submitFeedback` and `sendChat` mutations via `mergeSignals(assembled, ctx.result)`. Historical runs (pre-v2 Phase 0) keep working: every key is **optional** on `ReviewResultSchema`, so a legacy result with none of them parses cleanly.

12 · End-to-end sequence (upload → review → push)

A single trace of what happens from the moment an IC drops a transcript to the moment the ClickUp tasks land. Every v2 addition is annotated.



13 · Queues, streams & data stores

BullMQ queues (meetsy-api)

Name	Purpose	Options	Phase
<code>analysis</code>	Transcript → candidate tasks (pipeline)	—	v1
<code>meetsy-kb</code>	KB embed + status broadcast	—	v1 (Phase 2)
<code>meetsy-kb-docs</code>	PDF/SOP ingest → chunk + embed	—	v1 (Phase 2b)
<code>meetsy-push-retry</code>	Retry failed ClickUp pushes	<code>attempts:4</code> , exp backoff <code>2000ms</code> , <code>removeOnComplete/Fail: 100</code>	v2 Phase 2

Redis usage (three roles)

Queue

BullMQ for the queues above. Workers + producers use separate Redis connections per the existing pattern.

Cache

`LearningCacheService` — key `meetsy:learning:snapshot:v1:{workspaceId}` , SETEX 3600. Hard-invalidated on `logFieldOverride` . Bounded staleness (1h) means a failed DEL is safe — nudges get worse if stale, they don't break.

Pub/sub

`LearningStreamService` uses PUBLISH `learning:{ws}` . Dedicated publisher connection; per-subscription subscriber connection; teardown on disconnect. Mirrors the `kbChannel` pattern.

SSE endpoints (event source)

Endpoint	Purpose	Payload	Phase
GET /runs/:id/stream	Pipeline progress + completion	{event, step, message}	v1
GET /workspaces/:id/kb/stream	KB build progress	{status, progress, message}	v1 Phase 2
GET /workspaces/:id/learning/stream	Near-gate / gate-passed toasts	{workspaceId, field, predicted, confirmed, count, at, kind}	v2 Phase 3

New endpoints — quick reference

Endpoint	Roles	Delivers
GET /workspaces/:id/runs	any authed	Paginated run list with derived pushStatus. Phase 0
GET /workspaces/:id/runs/search?q=	any authed	Full-text search via tsvector. Phase 1
GET /workspaces/:id/clickup/tasks/:taskId	any authed	Resolves an evidence chip to {title, status, assigneeName, url} . Phase 0
GET /workspaces/:id/learning/me	any authed (self)	Per-user 6-week accuracy/nudge digest. Phase 1
GET /workspaces/:id/learning/gate	any authed	{minCorrections, minAgreement, nearGateThreshold, fields} . Phase 3
GET /workspaces/:id/learning/patterns/:key/history	any authed	Pattern's timeline (up to 500 rows). Phase 3
GET /workspaces/:id/kb/tasks?cursor&filter	any authed	Keyset-paged distinct embedded ClickUp tasks. Phase 4
POST /runs/:id/push/retry	any authed	Retry failed pushes. Phase 2
GET /workspaces/:id/push/dead-letter	Owner/Admin	List unresolved dead letters. Phase 2
POST /workspaces/:id/push/dead-letter/:id/resolve	Owner/Admin	Hand-mark resolved. Phase 2
GET /workspaces/:id/ml-config	any authed	Current tunables + models + isDefault . Phase 5
PUT /workspaces/:id/ml-config	Owner	Persist tunables; invalidates learning cache. Phase 5
POST /workspaces/:id/ml-config/preview	Owner	Replay last-N runs against candidate.

		Phase 5
--	--	----------------

14 · UI journey — step-by-step verification

Use this section to walk the app end-to-end and confirm every v2 surface behaves as designed. Assumes a working workspace with a built KB and at least one past run. Take screenshots as you go if you'd like a visual record.

Test setup: log in via Clicksy (`app.localtest.me:3000/login` in dev) using an existing account; the session cookie carries across to Meetsy at `meetsy.localtest.me:3001` (or your equivalent). If `/auth/me 401s`, the cross-subdomain cookie is misconfigured — see the Phase 0 live verification section of `BUILD-JOURNAL.md` .

Journey A · Land + orient

- 1 Open `/` — should redirect to `/home` (client-side redirect via `router.replace` , brief spinner OK).
- 2 Verify the **sidebar** is visible on the left on `md+` screens with entries: Home, New meeting, Meetings, Knowledge base, Learning; Settings (▼) reveals Push + Tuning for Owner/Admin. Resize below 768px — sidebar collapses into a hamburger.
- 3 On `/home` confirm two-column layout: recent runs (up to 5) on the left, **learning digest card** on the right with three sparkline rows (Accuracy, Corrections, Nudge acceptance). If you're new, expect an empty state ("As you review runs...").
- 4 Click a run row — you should land on `/runs/:id` with the tabs visible.

Journey B · Meetings history + search

- 1 Navigate to `/meetings` . Verify pagination (20/page), status filter chips (All / Completed / Running / Failed), and the search input on the right.
- 2 Click **Completed** — URL updates to `?status=completed` without pushing history. Click **All** to clear.

3 Type a keyword from a past transcript in the search box — 300ms after you stop typing, URL updates to `?q=<term>` and the list re-renders. Confirm the empty-state icon adapts (`Search` when q is active; `ListChecks` otherwise).

4 Click the `X` in the search input — `q` clears; list re-loads.

Journey C · Evidence-first review (the big one)

1 Open any completed run at `/runs/:id`. Verify tabs at the top: **Overview / Push / Chat / Insights**. Default is Overview.

2 On each task card, evidence is **expanded by default** — no accordion click needed. Sections top-to-bottom: Duplicates → Suggested fields → Owner → Learning nudge → Similar (if `neighboursByTask` is present).

3 Click any red or amber **duplicate chip** (`CU-01H...`) — a right-side `TaskDetailSheet` opens with title, status, assignee, updated timestamp, and "Open in ClickUp ↗" link. If the task predates the KB, the sheet reads "we don't have this task's title." `Esc` closes.

4 Under Owner, click any of the `evidenceTaskIds` chips. Same sheet, different task.

5 Try `j` and `k` — focus rings move between task cards; the card smooth-scrolls into view. `↓` / `↑` work too. Type in an input field first — `j` / `k` stop firing while you're typing.

6 If you see a **Learning nudge**, confirm it references either `assignee` or `sprint`. If your KB is small, expect no nudges — check the Learning page to see if patterns are building up.

7 Deep-link check: append `#push` to the URL and reload — the Push tab is now active. Same with `#chat`, `#insights`. Unknown hash → back to `overview`.

Journey D · Push + retry

1 Switch to the **Push** tab. Confirm the workspace is **push-configured** (target list picked) or you'll see a "not configured" hint.

2 Push a task (or use a run that has already been pushed). Confirm every pushed row shows a `pushed` pill and a link out to the created ClickUp task.

3 If any push failed, a **"Retry failed (N)"** secondary button appears in the header row. Click it — the toast should confirm the enqueue; ~3 seconds later the page reloads and failed rows have flipped to `pushed` (or stayed `failed` with a dead-letter row for later).

4 (Owner/Admin only) Query `GET /workspaces/:id/push/dead-letter` — an unresolved list shows every push that exhausted 4 attempts. `POST /:id/resolve` hand-marks with your `userId`.

Journey E · The learning loop, made visible

1 Navigate to `/learning`. Three sections stack: **Active** (patterns the loop currently gates), **Building up** (near-gate patterns with a `count / minCorrections` progress bar), and **Coverage** (per-field predictions seen / corrections / nudge accepts / unresolved).

2 Click any pattern row in **Building up** — `PatternHistorySheet` opens on the right with the pattern's aggregate stats + a chronological entry list.

3 (Optional, requires a workspace with a pattern one correction away from gating) Trigger a correction that pushes a pattern to `count === MIN_CORRECTIONS - 1`. Watch for a **Sonner info toast**: "One more correction and $X \rightarrow Y$ will start nudging." Push one more — the toast should upgrade to a green success: " $X \rightarrow Y$ is now nudging."

4 Confirm the "Predictions you changed" and "Suggestions accepted" labels — no `rawOverrideRate` or `nudgeAcceptanceRate` jargon visible to the user.

Journey F · Knowledge base — one route to rule them all

1 Navigate to `/kb`. Above the tabs, the **KB status card** is visible (idle / building / ready). If the KB isn't ready, a banner appears with a build panel (Owner/Admin) or a "ask an Owner/Admin" message (Members).

2 Cycle the tabs: **Overview** (facts summary) · **Tasks** (paginated, filter box, click a row → `TaskDetailSheet`) · **Documents** (upload, list, delete for Owner/Admin) · **Search** (debounced `/kb/search`, results as clickable cards, `?q=` in URL) · **Rebuild** (Owner/Admin only — `KbBuildPanel`).

3 Hit `⌘K` (or `Ctrl+K`) — the command palette opens overlay-style. Under **Go to:** Home / New meeting / Meetings / Learning / Knowledge base (+ Push for Owner/Admin). Under **Search knowledge base:** type a query, 250ms later hits appear; selecting one navigates to `/kb?tab=search&q=<query>`.

4 **Verify `/onboarding` is retired** — type `/onboarding` into the URL bar. It should 404 (page.tsx was deleted). `/settings/kb` should redirect to `/kb?tab=rebuild`.

Journey G · Owner tuning + preview

1 As Owner, navigate to `/tuning`. Sections stack: Duplicate detection / Similarity / Gate / Novelty / KB. Every field renders as a numeric input with the current value.

2 Fields with a **"Not applied yet"** chip are STORED but not consumed by runtime code (Phase 5 honesty). Fields WITHOUT the chip (dup bands + gate values) are LIVE.

3 Change `dupFlag` from 0.72 to 0.80. Click **Preview** — a right-side sheet opens with per-run delta (baseline vs candidate flag/suggest counts, and how many tasks would have re-classified) for the last 10 runs.

4 Confirm the **skipped-fields note** at the bottom of the preview lists the tunables that aren't in preview (simFloor, rrfK, etc. — non-replayable without re-embedding).

5 Click **Save** — Sonner success toast, page re-fetches. Trigger a new run to prove the runtime picked up the new bands.

6 As Member, navigate to `/tuning` — the page loads read-only (inputs disabled; no Save or Preview buttons). Sidebar doesn't show a Tuning entry for Members.

Journey H · Dark mode + a11y

- 1 Bottom-right of the sidebar (or top-right on mobile) — click the **theme toggle**. Cycle Light / Dark / System. Every route from Journey A–G should render correctly with no white-on-black cards.
- 2 Tab from the top of any page — the first focus stop is "**Skip to main content**". Press Enter — focus jumps to `<main>`.
- 3 macOS: System Settings → Accessibility → Display → **Reduce Motion**, then reload. Sheet slides, toast slides, animate-in transitions — all visibly gone.
- 4 Resize the viewport below 768px — sidebar collapses to a hamburger, review page task cards stack flush-left, long task titles wrap instead of overflowing.

15 · Success-criteria checklist

Direct from the v2 plan (§6). All nine criteria are met at the code + verify level as of commit `7884b32` on branch `feat/meetsy-phase0`.

- ✓ 1. A user lands on `/home` after login, sees their recent runs, and can navigate to any past run without URL memory. Phase 1 · PR-D
- ✓ 2. Every prediction chip on a task card is **clickable** and opens a side sheet with the past ClickUp task's title/status/owner/URL. Phase 2 · PR-K
- ✓ 3. `assignment.ranked[]`, `evidenceTaskIds`, and `FieldPrediction.candidates[]` are **visibly rendered**, not tooltip-only. Phase 2 · PR-J
- ✓ 4. A user can see patterns **building up** in the learning loop (2/3 corrections, near-gate), not just active nudges. Phase 3 · PR-O
- ✓ 5. The learning loop covers both `assignee` and `sprint`. Phase 3 · PR-L
- ✓ 6. An Owner can adjust duplicate bands / sim floor / gate values from `/tuning`, preview the impact against last-N historical runs, and save without a deploy. Phase 5 · PR-W · dup bands + gate live; other fields stored + previewed
- ✓ 7. `/kb` is one route with search, browse, upload, and rebuild all in tabs. Phase 4 · PR-Q
- ✓ 8. Feedback + chat mutations no longer strip evidence from `AnalysisRun.result`. Phase 0 · PR-A + Phase 2 extension for neighbours
- ✓ 9. Dark mode works end-to-end; keyboard traversal (j/k) works on the review page. Phase 6 · PR-AA + PR-BB

Test suite growth (per phase)

V1 → PHASE 0 217 → 235 +18 tests	PHASE 0 → 1 235 → 246 +11 tests	PHASE 1 → 2 246 → 262 +16 tests	PHASE 2 → 3 262 → 293 +31 tests
PHASE 3 → 4 293 → 300 +7 tests	PHASE 4 → 5 300 → 313 +13 tests	PHASE 5 → 6 313 → 313 frontend-only	FINAL TOTAL 313 56 suites, all green

Honest deferred list

Not everything is done. These are documented deferrals, not accidental gaps:

- **Non-replayable tunables** (`simFloor` , `minQualifying` , `closedWeight` , `rrfK` , `novelMaxSimCutoff` , `linkMinSim` , `embedBatch` , model routing) — stored in `WorkspaceMlConfig` , "Not applied yet" chip in UI, one-file refactor each to consume.
Phase 6+
- **meetsy-ml-preview BullMQ queue** — Phase 5 preview is synchronous JSON math today; queue is documented as ready to wire when preview grows heavier. If needed
- **Full a11y audit** (axe/WCAG-AA sweep) — Phase 6 hit landmarks, focus, skip link, reduced motion, `aria-current` , `role="alert"` . A formal audit is not run. Follow-up
- **Palette recent-search history** (local-storage backed) — Phase 6 polish. Nice-to-have
- **Bottom-drawer TaskDetailSheet on mobile** — reconsidered and skipped (Radix Sheet unmount during resize is a risk; the right-side sheet already collapses to full width on phones).
- **Cross-run task linking** ("this task in run A is a continuation of task in run B") — deferred at plan level, not scoped for v2.
- **Migrations UNAPPLIED at commit time:**
`20260718150000_meetsy_v2_phase1_run_search` ,
`20260718200000_meetsy_v2_phase2_push_dead_letter` , and Phase 0's
`WorkspaceMlConfig` / `AnalysisRunSnapshot` migration all ride `prisma migrate deploy` on next deploy.

16 · Verification results & ops appendix

All-green verification (2026-07-19)

Target	Command	Result
meetsy-web typecheck	<code>pnpm --filter @ma/web typecheck</code>	PASS
meetsy-web lint	<code>next lint</code>	PASS · 0 warnings · 0 errors
meetsy-api typecheck	<code>pnpm --filter @ma/api typecheck</code>	PASS
meetsy-api jest	<code>npx jest</code> (in apps/meetsy- api)	56 suites · 313 tests · 5.918s
Working tree	<code>git status</code>	clean
Branch	<code>git branch --show- current</code>	feat/meetsy-phase0
HEAD	<code>git rev-parse --short HEAD</code>	7884b32 — feat(meetsy): v2 Phase 6 — cross-cutting UX polish

Why next build is skipped. Per the `meetsy-web-next-build-dev-footgun` memory, running `next build` against a live `next dev` server's shared `.next` state breaks every route in dev. `typecheck + lint + jest` is the sanctioned verification path here.

Running the project — from scratch (one-time, or after a full wipe)

```
## 1 · Machine prerequisites
brew install pnpm # lives at /opt/homebrew/bin/pnpm, survives nvm s

## 2 · Node version – reads .nvmrc = 22.13
cd /path/to/clickup-sync
nvm install # installs 22.13.x if missing
nvm use # activates for this shell
nvm alias default 22.13 # optional: make it your global default

## 3 · Env file (once per clone)
cp .env.example .env # fill in real values

## 4 · Install deps – postinstall hooks run BOTH prisma generate calls automatically
pnpm install

## 5 · Start Docker services
```

```

npm run dev:deps                                # docker compose up -d postgres redis

## 6 · Apply DB migrations
npm run prisma:deploy                            # Clicksy (public schema)
psql -f apps/meetsy-api/prisma/grants.sql        # first time only – creates r
pnpm --filter @ma/api exec prisma migrate deploy # Meetsy (meetsy schema)

## 7 · Boot the whole platform
npm run dev:platform                            # clicksy-api :3000 · clicksy-web :5173 · meetsy-

```

Running the project — every subsequent boot

```

cd /path/to/clickup-sync
npm use                                # 22.13 from .nvmrc (skip if it's your default)
npm run dev:deps                       # containers up (no-op if already running)
npm run dev:platform                   # all four apps in one window

## If a git pull changed the schema or deps:
pnpm install                          # postinstall regenerates Prisma
npm run prisma:deploy                  # apply new Clicksy migrations
pnpm --filter @ma/api exec prisma migrate deploy # apply new Meetsy migrations
npm run dev:platform

```

Quality checks before every commit

```

## Clicksy backend (root)
npm run lint
npm run test
npm run build

## Meetsy – sanctioned verify path (next build deliberately skipped per meetsy-web-next-l
pnpm --filter @ma/api typecheck
pnpm --filter @ma/api test
pnpm --filter @ma/web typecheck
pnpm --filter @ma/web lint

```

Common gotchas — quick reference

Symptom	Fix
command not found: pnpm after nvm deactivate	brew install pnpm — puts it at /opt/homebrew/bin/pnpm , immune to nvm state.
Cannot find native binding @rolldown/binding-*	Node was too old at install time. Switch Node → rm -rf node_modules → pnpm install . The repo's .npmrc now sets engine-strict=true , so this fails loudly at install instead of silently skipping.
200+ TS errors like Property 'meeting' does not exist on type 'PrismaService'	Prisma client wasn't generated. The postinstall hooks in root + apps/meetsy-api/package.json handle this automatically. If somehow bypassed: pnpm --filter @ma/api exec prisma generate + DATABASE_URL=postgresql://placeholder pnpm exec prisma generate --config ./prisma.config.ts .
Vite refuses to start ("Node.js version 22.13+ required")	nvm use in the repo dir picks up .nvmrc = 22.13 .
pnpm 11 refuses to boot on Node 22.12	Homebrew's pnpm 11.15+ needs Node ≥22.13; the .nvmrc and engines.node both say 22.13. nvm install && nvm use .

Cross-subdomain cookie test (from the Phase 0 live-verify)

Use *.localtest.me (all subdomains resolve to 127.0.0.1) so a parent-domain cookie genuinely crosses origins. localhost ports alone are different origins and won't share cookies.

```
## Sign up on Clicksy
curl -i -c cookies.txt -H "Content-Type: application/json" \
  -d '{"email":"o@localtest.me","password":"...","name":"...","orgName":"..."}' \
  http://app.localtest.me:3000/api/auth/signup

## Confirm Meetsy accepts the shared session
curl -i -b cookies.txt http://meetsy.localtest.me:3010/auth/me
## → 200 {"userId":"...","orgId":"org_seed","role":"OWNER","email":"o@localtest.me","isMeetsy":true}
```

Environment variables that matter for v2

Variable	Purpose
<code>MEETSY_DATABASE_URL</code>	Meetsy DB URL, points at the <code>meetsy</code> role, <code>?schema=meetsy</code> so <code>_prisma_migrations</code> lands correctly.
<code>COOKIE_DOMAIN</code>	Optional parent-domain scope for the session cookie so it crosses subdomains.
<code>NEXT_PUBLIC_CLICKSY_LOGIN_URL</code>	Where meetsy-web bounces 401s to.
<code>AZURE_OPENAI_*</code>	Unified v1 endpoint + api-key; chat + embeddings on one resource as of 2026-06-29.
<code>REDIS_URL</code>	BullMQ + LearningCache + LearningStream pubsub.

Where to look when something breaks

Symptom	First place to look
Signals disappear after feedback / chat	<code>analysis.service.ts:227</code> (<code>loadRunContext</code>) and <code>mergeSignals</code> at file bottom.
Chip click does nothing	<code>TaskSheetProvider</code> mounted in the subtree? <code>useTaskSheet()</code> hook fires?
Learning toast never fires	Redis pubsub reachable? <code>LearningStreamService</code> log; <code>useLearningStream</code> mounted in <code>SignedInShell</code> ? Was the correction nudge-influenced (won't fire)?
Push retry stuck	<code>PushDeadLetter</code> table — if a row exists with <code>attemptsMade=4</code> , ClickUp genuinely refused four times; inspect <code>errorMessage</code> .
Tuning save has no runtime effect	Two possibilities: (a) field is "Not applied yet" (see stat above the input); (b) <code>LearningCache</code> not invalidated — check <code>MlConfigService.upsert</code> logs.
Dark mode reverts on hard reload	<code><html suppressHydrationWarning></code> present? <code>ThemeProvider</code> 's <code>defaultTheme="system"</code> + <code>enableSystem</code> both set?
<code>j / k</code> doesn't scroll	Are you focused inside an input? The hook guards against it. Are there any <code>[data-task-anchor]</code> elements in the DOM?
Full-text search returns no matches	Migration <code>20260718150000_meetsy_v2_phase1_run_search</code> applied? Postgres 12+? Generated column populated (should be automatic on INSERT/UPDATE)?

Source-of-truth files, in read order

1. `docs/superpowers/plans/2026-07-18-meetsy-v2-plan.md` — the umbrella plan.

2. `docs/superpowers/specs/2026-07-18-meetsy-v2-phase{0..6}-*-design.md` — one spec per phase, matches the six phase-cards above.
3. `docs/meetsy/BUILD-JOURNAL.md` — append-only per-PR journal with verify tables (this is where every claim in this guide is grounded).
4. `apps/meetsy-api/prisma/schema.prisma` — final data model.
5. `packages/shared/src/review-result.ts` — the `ReviewResultSchema` that solved the signal-loss bug.
6. `apps/meetsy-web/app/{home,meetings,runs/[runId],learning,kb,tuning}/` — all v2 routes.